Development Plan
Software Engineering

Team 22, Ghoti
Hemal Rajput
Gurmanjot Minhas
James Araiche
Lily Porter
Sarah Dorfman

Table 1: Revision History

Date	**Developer(s)**	**Change**
September 28, 2026	All	Initial Revision for Deliverable 1

The following document outlines the development plan for the **Ghoti** capstone project. It contains all administrative project details including, confidentiality, IP, team member information and communication protocols. It also includes project technical plans and standards, including the Proof of Concept design, expected technologies and details on AI usage.

1 Confidential Information?

During development, there is no confidential information being used. Machine learning training data will be gathered from publicly available resources. However, when the product is in use by patients, their data will be private medical data. This means that the final product must be PHIPA compliant.

2 IP to Protect

The product will use a non-exclusive license, thus ownership and control of the product belong to Team Ghoti. Ghoti will be able to grant licenses of the product to any company it chooses. Any royalties and revenues generated by the product will be equally distributed amongst Ghoti's licensees under the terms of each non-exclusive license.

3 Copyright License

Our team is adopting the GNU General Public License (GPL). The license is available in the LICENSE file at the root of our repository.

4 Team Meeting Plan

In-person team meetings will be held weekly on Thursdays from 3:30pm to 4:20pm. The purpose of these meetings will be to discuss upcoming deliverables, progress updates, and potential blockers. They will also be used to consolidate any questions that team members may have for the instructional team and project supervisors.

There will also be biweekly meetings held with the project supervisors on Fridays from 9:00am to 10:00am. They are planned to be in-person but may be shifted to virtual Microsoft Teams meetings based on availability. These will be used to ensure the project is in line with the advisors' requirements, and to address any questions that the development team might have.

The meeting chair (Sarah Dorfman) will prepare an agenda for the main topics of discussion and distribute it to the team at least one day prior to any team meetings to give all members time to prepare. All members are encouraged to take notes during the meeting. After the meeting, the team will collectively

assign action items to each other. If there is a supervisor meeting planned for that week, the team will also plan a corresponding agenda.

5 Team Communication Plan

The team will use Microsoft Teams as the main avenue of communication. All group members are required to keep notifications on so that emergencies can be addressed in a timely manner. It will be used to discuss ideas, share files, and join informal meetings when needed.

To supplement these informal discussions, the group will also maintain Github issues to track tasks in a more formal manner. Issues should clearly describe the task, come with an appropriate label, and be assigned to the relevant people. This is not necessary for issues concerning lectures, or team meetings, as it is assumed that all team members attend these.

Communication with project supervisors will be done through McMaster Outlook emails. Based on the nature of the interaction, communication with the instructional staff may be done through McMaster Outlook emails, or through the discussion boards on Avenue to Learn.

6 Team Member Roles

To ensure the team works collaboratively and effectively, we have defined unique roles for each member. Each role has its own clearly defined tasks and responsibilities that reflect the team's goals and individual strengths of team members. While each team member will have a primary area of focus, these roles are not fixed and may evolve as the project progresses and team members rotate responsibilities:

- **Project Manager & CI/CD/Admin Lead:** This role oversees and manages the project timeline, team organization and progress. The project manage will maintain the project's the CI/CD pipeline to ensure LaTeX documentation compiles and deploys correctly. They will also manage repository permissions, create issues, track team progress, and organize milestones and deadlines.

- **Team Liaison:** This role is the main point-of-contact between the team and course instructors. They are responsible for submitting/creating pull requests for deliverables, and facilitating communication of feedback and comments between the instructor and the group.

- **Requirements Expert:** This role leads communication with project stakeholders. They will lead the gathering of system and stakeholder goals and lead the process of translating them into clear requirements to support the writing of the Software Requirements Specification (SRS) documentation, ensuring traceability between stakeholder goals, system requirements and the final design.

- **Systems Architect/Development Lead:** This role will ensure that the high-level system architecture is followed and oversees the core implementation of the software. They will ensure the team adheres to the defined architecture and design principles and ensure that the system is designed with extensibility to accommodate changes and project expansion.
- **Machine Learning (ML) Lead:** This role is responsible for overseeing the design, implementation, and evaluation of the project's machine learning components. They will be the primary manager/liaison for datasets, model training, and performance evaluation.
- **User Interface & Experience (UI/UX) Lead:** This role is responsible overseeing the design and implementation of the user interface. They will be the primary manager/liaison for the creation of wireframes, prototypes and the project's adherence to human-centred design.

7 Workflow Plan

Stakeholders and developers will work closely together on creating this project. Thus, it is cruicial to use a workflow that simplifies the development cycle, allows for rapid feedback from stakeholders, and allows for quick iterations on the product. The quicker developers are allowed to iterate, the faster they can tune the product to stakeholder needs.

The workflow selected for this project is the GitHub workflow due to its simplicity and efficiency. The GitHub workflow dictates that each feature branch will be merged directly into the main branch, and product releases will come from the main branch. Overhead branches are not required for the GitHub workflow, minimizing the amount of branch management a developer has to do. Production releases can be created quickly after each feature branch is merged, thus allowing for rapid feedback from stakeholders and allowing developers to make quick, iterative changes to make stakeholders happy.

For traceability, GitHub issues created with GitHub Projects will be used to track each feature. A developer will assign themselves to an issue on the project board and create a feature branch to work on it. Once a feature branch is ready to be merged into the main branch, the developer assigned to the issue will create a pull request. For each issue, at least one other team member must review it before approval is given to merge the associated feature branch into the main branch. Once the feature branch is merged, the developer assigned to the issue must close it and set its status to "done." Preferably, each pull request should contain a maximum of 500 changes to lines of code to minimize the workload on reviewers for each individual pull request.

Lastly, GitHub Actions will be used to create a pipeline for continuous integration and continuous development. An automatic runner for format checking, unit testing, and vulnerability checking will be part of the pipeline. Since the project is using a model that requires training and/or fine-tuning, benchmark testing, regression testing, and model latency checks will also need to be added

to the pipeline to ensure thresholds for the product are met. Other checks may be added to the pipeline as needed.

8 Project Decomposition and Scheduling

GitHub projects will be used for tracking lectures, meetings, product progress, and project deliverables. Below is the information we will use GitHub projects to keep track of.

- Lectures: lecture name, date and time of the lecture, team members in attendance, and questions for end of lecture.
- Meetings: date and time of the meeting, members in attendance, agenda for the meeting, and action items to work on after the meeting has concluded.
- Product Progress: Issue number, feature description, tests changed/added, and before/after screenshots.
- Project Deliverables: Issue number, deadline, and contributors paired with assigned deliverable sections.

Inside the project, meeting information can be found under "meetings" and lecture information can be found under "lectures". Product progress and project deliverables can be found under "Backlog", "Ready", "In Progress", "In Review", or "Done" based on its status. Product progress and project deliverables will move through each of these stages sequentially and at each stage will be reviewed by at least one team member.

Link to the project can be found Here

9 Proof of Concept Demonstration Plan

At a high level, this application will listen to child speech, process it into its phonetic representation, and analyze this to give feedback to SLPs. The greatest risks associated with this lie in the intermediate audio processing stage:

1. **Converting Child Speech To Phonetic Representation -** Since children are in a developmental phase, their speech patterns vary significantly when compared to adults. Therefore, it may be challenging to ensure an acceptable accuracy rate when converting child speech into phonetic representations.
2. **Ensuring An Acceptable Processing Time -** For this application to be viable in a clinical setting, it will need to give real time feedback to the SLP during an appointment on their computers. As per the project supervisors, existing child speech interpretation models are quite slow, so it may be difficult for this one to process the data fast enough to give the

clinician feedback in less time than it would have taken for them to reach a deduction themself.

To ensure these risks can be overcome, a suitable training dataset must be found online. To ensure speech to phonetic conversion accuracy, there must be sufficient data to properly train the model. It is important to note that this data must be publicly available to comply with PHIPA patient confidentiality guidelines.

Once this data is sourced, we will need to develop the actual model that takes child speech and converts it into a readable format with the corresponding phoneme representations. If this model meets the established accuracy benchmarks, it will be feasible to continue designing this application for pediatric speech purposes. Moreover, it is fair to assume that speech therapy offices will use typical office computers that are not equipped with powerful hardware. Therefore, it is necessary to show that the model can meet the benchmark processing time on a device of similar specifications.

10 Expected Technology

10.1 Languages and Constraints

- **Python**: speech processing, model evaluation and training, and application backend.
- **JavaScript:** user interface.
- **SQL:** data storage and access control.
- **Git, GitHub, and GitHub Projects:** course requirements and will be used for version control, issue tracking, code review, and planning.
- The application is web-based so it will run on devices that a typical clinic would have, i.e. a desktop.

10.2 Libraries and Frameworks

- **Backend**: an Python web framework (e.g., FastAPI) with data validation (e.g., Pydantic).
- **Frontend**: a JavaScript framework (e.g., React), an API to capture audio from the browser.
- **Data**: a relational database (e.g., PostgreSQL) with row-level access control, hosted or self-hosted.
- **Speech and machine learning**: PyTorch, the Hugging Face Transformers library, and standard audio-processing libraries.

10.3 Pre-trained Models and Training

- We expect to start from pre-trained speech models (e.g., wav2vec2) and existing phone recognizers, to train a model, if the existing solutions are found to not be good enough.
- Training data will be publicly available child-speech corpora with phonetic annotations
- Both existing commercial and open-source models are evaluated and are a fallback if a trained model does not meet the agreed targets

Components we expect to implement ourselves.

- comparison of recognized phones against the target sound
- an evaluation harness for accuracy, speed, and regression testing
- the application: practice interface, curriculum, and clinician dashboard.

10.4 Development Tooling

- **Linting and formatting**: Ruff for Python; Prettier for JavaScript.
- **Unit testing**: pytest for Python; Vitest for JavaScript.
- **Continuous Integration**: Pull requests must have a reviewer; branch protection blocking tests that fail; unit testing to prevent breaking the application on code that compiles on merge;

10.5 Performance Measurement

- **Accuracy**: phone error rate against reference transcriptions; precision and recall of substitution detection per target sound; agreement with speech-language pathologist annotations.
- **Speed**: real-time factor (processing time divided by audio duration) and end-to-end latency, measure at each model's size to map the accuracy-versus-latency tradeoff.
- **Regression testing**: a fixed held-out evaluation set is re-run after every training run or model change, and a change is accepted only if results stay within agreed thresholds.

10.6 Areas Requiring Additional Training

- Phonetics and speech sound disorders, with our SLP advisor.
- Fine-tuning and evaluating speech models.
- Database access-control design.

11 Use of AI and LLMs

Our team will use LLMs to support brainstorming, application development, and documentation. Tools such as Claude and ChatGPT may be used to explore design approaches, explain concepts, and write code. In the coding process, agents will also be implemented to improve workflow and follow best practices. Team members will make and take responsibility for all project decisions and submitted work.

We will review any generated code before including it in the application. We will check that it meets the project requirements, follows our design, and works as intended by running relevant builds, checks, and tests. Any output that cannot be verified or does not meet these checks will be revised or discarded.

12 Coding Standard

- **Python**: PEP 8, enforced with Ruff for linting and formatting
- **JavaScript**: the Airbnb JavaScript Style Guide, enforced with Prettier.
- **Commits and pull requests**: descriptive commit messages, one pull request per feature or fix, and at least one reviewer approval before merging to the main branch.
- These standards are enforced automatically through GitHub Actions on every pull request; a failing lint or format check blocks merging.

13 Appendix — Generative AI Use Disclosure

All writing was done by the team members by hand. All research and facts were obtained through academic sources, or by consulting the project supervisors unless explicitly stated below.

13.1 AI Tools Used

Table 2: List of AI Tools Used

Tool Number	Name	Model
1	Claude	Sonnet 5
2	ChatGPT	GPT 5 Thinking Mini

13.2 How and Where Used

Table 3: Usage Details for All AI Tools Used

Tool Number	Location	Details
1	Bibliography	Used to insert a reference to the references.bib file.
2	Problem Statement and Goals; (1.3) Stakeholders	Instructed the AI to think from the perspective of a patient's parent when assessing stakeholders: highlighted privacy concerns as another consideration for this stakeholder.

13.3 Verification of AI-Generated Content

As none of the AI usage was for factual content, verification was very simple. For tool 1, we simply verified the resulting citation for correctness in the final output. For tool 2, it was used as more of a brainstorming utility, so only unconsidered elements that made logical sense were included.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. **Why is it important to create a development plan prior to starting the project?** Much of the lost time in any project, especially a group project, is due to disorganization and poor coordination. Developing that organizational framework ensures that the team is on track to complete the project in a timely manner. The value of development planning rather than jumping straight into the code, is ensuring that more of the thinking is done sooner. The later work is instead coordinated implementation. Communication is one of the most challenging parts of engineering. We can ensure that we all know our expectations of the project, and we can resolve miscommunication early on.

2. **In your opinion, what are the advantages and disadvantages of using CI/CD?** CI/CD automates much of the menial labour in software development. This lets the team focus on more critical aspects of the work instead of getting bogged down by the friction of testing and deployment. It is also a significant time saver. Many mistakes, which may be missed by human reviewers, will be caught by the CI/CD pipeline. This means reviewers can focus on architectural and design details as opposed to spending time on trivial things, such as naming conventions.
   On the other hand, the main disadvantage of implementing CI/CD is the initial time spent setting it up. This is especially true for us, since none of us have implemented CI/CD in our own projects, even though we may have worked in codebases that had CI/CD pipelines during our co-ops. Spending too much time on the perfect CI/CD pipeline is time lost on the features of the project.

3. **What disagreements did your group have in this deliverable, if any, and how did you resolve them?** We had some disagreements on the Expected Technology section, specifically about the use of Cloud-based and web solutions for our project. This project will likely require storing and analyzing the user's medical data, which means we need to consider the ethics and security of our solution. Any web-based application

would require considering cybersecurity, and any Cloud-based databases would require detailed inspections of the provider's security and privacy policies. However, these approaches would significantly ease the technical development of our project, allowing us to focus on meeting our goals. We had to spend some time considering the trade-offs of each approach and come to a compromise to move on to other aspects.

Appendix — Team Charter

External Goals

The team's goals for this project revolve around how useful our product will be to stakeholders. By the end of the capstone, Ghoti is hoping to have created a tool that its supervisors and stakeholders will be using and experimenting with in their labs and clinics. Ghoti also hopes that if the product is trusted, it will be an asset for team members to acquire future employment and bode well for final grades in the capstone course.

Attendance

Expectations

Team members are expected to attend all meetings at assigned times. Meetings are booked solidly in advance every Thursday from 3:30pm to 4:30pm. If no cancellations or prior exceptions have been given, all members are expected to attend.

Acceptable Excuse

Acceptable excuses for missing meetings:

- Family/personal issues or events
- Personal events (given 1-3 days' notice depending on the current capstone situation)
- Personal happenings with school (e.g., midterms, group assignment meetings, etc.) (Must be given 1-3 days' notice depending on the current capstone situation)

Should a team member have anything that is required for them to contribute to in a meeting, they should contribute as much as they're able to before they miss the meeting.

Acceptable excuses for missing deadlines

- Family/personal issues or events
- Personal happenings involving school (e.g., midterms, group assignment meetings, etc.) (Should notify the group ASAP if you realize you are going to miss a deadline due to any school factors).

Missing one deadline is not the end of the world; missing numerous is a pattern and will be considered a problem that will need to be brought up with Dr. Smith. Team members should reach out as early as possible if they realize they are going to miss a deadline so the team can act quickly to help.

In Case of Emergency

- Miss a meeting: Team members should contribute as much as they can before the meeting; if required, team members will designate work assigned in the meeting to the missing member and will notify them of this once the meeting is finished.
- Can't meet a deliverable: Team members should contribute what they can but let the team know of the situation ASAP. The team will reassign work to complete the deliverable on time. Life happens, but poor communication or repeated failures may involve team discussion or discussions with Dr. Smith.

Accountability and Teamwork

Quality

- Quality of Team Meetings: Team members are expected to show up with a general idea of what will be on the agenda for any given meeting. If there is any material or information team members need to have prepared before the meeting, they are expected to be prepared with said materials. Should a team member find themselves unable to properly prepare for a meeting, they should notify their team members as soon as possible and work with the team to find a solution. Situations like these can be forgiven as long as they are communicated ahead of time and not part of a greater pattern of failing to deliver. If a team member is assigned to take meeting notes, the notes should be concise and cover all key discussion points.
- Quality of Deliverables: Each team member should complete their assigned portions of deliverables (barring accepted excuses), focusing as much as they can on producing quality work. Deliverables should be reviewed by each team member, and inconsistencies/issues should be brought up with the individual(s) responsible for creating them. Team meetings or discussions may be needed if the inconsistencies/issues are contested topics that cannot be resolved solely by the involved parties.

Attitude

Each team member should bring the attitude of wanting the team to succeed in the goals it set out to achieve. The actions that team members take should be meant to uplift the project and team as a whole and not tear each other down. Specifics are found in our code of conduct and conflict resolution plan:

- Code of Conduct:
  - Team members should treat each other with respect and dignity. No personal attacks or insults are allowed in team discussions.
  - Having fun/joking around is fine as long as it doesn't degrade the quality of a meeting or cause discomfort in team members.

  - Team members should communicate quickly and often; the quicker issues are communicated, the quicker they can be resolved.

- Conflict Resolution Plan:
  - Issues between team members should be discussed individually first; calling each other out in team meetings shouldn't be the go-to method of discussing issues unless it is warranted.
  - Should no resolution be found between individuals, conflicts will be raised to the team level for discussion.
  - Should team discussions fail, the issue will be brought to the TAs/Prof for resolution.
  - Conflicts of a serious or harmful nature should be directly brought up to the professor for the safety of the parties involved.

Stay on Track

The team will use tentative deadlines to keep each other on track for team deliverables. During team meetings, we will discuss current progress, roadblocks, and upcoming deadlines to ensure everyone is on the same page with the work they have been assigned. A team member who is confused about the work expected from them should immediately reach out to the group for clarification. Team wins should be recognized throughout the capstone project's development, and shoutouts should be given to team members who are doing well in their work. If there are team members who are performing below expectations, the team should come together to address the issues hindering them. If necessary, a plan should be made for the team member that outlines steps for improvement. Should a team member fail to meet two deadlines for project deliverables, frequent or daily check-ins will be required to keep them on track and monitor their progress. Should failures to meet project deliverables continue, discussions with the Dr. Smith will be arranged for next steps.

Attendance will be taken at each scheduled team meeting and will be measured for metrics. Below 80% attendance to meetings, team members will be flagged after each meeting, below 60% team discussions will occur and improvement plans might be created. Below 50%, a meeting with the TA will be arranged. Below 30%, a meeting with Dr. Smith will be arranged.

The team will divide work equally as it relates to GitHub issues. If everyone is doing their assigned share of GitHub issues, it will be considered as the work being equally shared. If the team finds out that there is an unfair balance of work given to a team member, the team will reconvene to redistribute tasks. Should the team meet targets early we will all celebrate by going out for lunch.

Team Building

The team will go out for a trivia night or a similar team-building activity at least once per semester.

Decision Making

- Individual decisions: Should decisions need to be made at a lower level, at least one to two other people should discuss available options and come to a consensus. If a consensus can't be reached, escalate to the whole team, where a majority vote will result in a decision being made.
- Team decisions: These should be immediately handled at the team level and will be sorted out with a majority vote resulting in a decision being made.
- Any team member not present at a meeting where a decision is made should be notified of the decision and given the opportunity to provide feedback. If the team members involved in the decision feel that the feedback is warranted, they will reconvene to discuss the feedback and make a final decision.